TECHNICAL DOCUMENTATION

Project: RAG-Based Customer Support Assistant

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a sophisticated framework designed to improve the reliability of Large Language Models (LLMs). It works by fetching relevant information from an external, authoritative knowledge base (such as technical manuals or policy documents) and providing that context to the LLM to ground its response. This process ensures that the generated output is based on facts rather than the model's static training data.

Why is it Needed?

Large Language Models, while powerful, suffer from information cut-offs and "hallucinations." In a customer support context, providing incorrect information regarding company policies or technical instructions can be detrimental. RAG mitigates this risk by forcing the model to cite and utilize specific document segments, thereby ensuring accuracy, transparency, and the ability to update the system's knowledge simply by adding new documents to the database.

Use Case Overview

The primary use case is a technical customer support bot. The system ingests PDF manuals, understands user queries, retrieves the exact instructions required, and formulates a response. Critically, it includes logic to detect when a user is frustrated or when the documentation does not contain the answer, escalating the query to a human agent.

2. System Architecture Explanation

The high-level design (HLD) follows a modular, decoupled approach to separate long-running tasks from real-time user interactions. The architecture is composed of two primary sub-systems: the Ingestion Pipeline and the Inference Graph.

Component Interactions

- **Frontend (React):** Acts as the user touchpoint for document management and chat. It maintains the session state and observes escalation flags to modify the UI layout.
- **Backend (FastAPI):** Provides RESTful endpoints for file uploads and query processing. It serves as the gateway to the LangGraph execution engine.
- **Vector Store (ChromaDB):** Serves as the system's semantic memory, storing multi-dimensional representations of document chunks.
- **Inference Engine (Groq/Llama 3.1):** Performs both the logical classification of user intent and the final synthesis of supported answers.

3. Design Decisions

Decision Component	Strategy Applied	Rationale
Chunk Size	500 Tokens / 50 Overlap	Optimizes context window utilization while preserving semantic boundary integrity.
Embedding Strategy	all-MiniLM-L6-v2	Chosen for low-latency CPU inference and high performance in sentence similarity tasks.
Retrieval Approach	Top-K Cosine Similarity	Efficiently identifies the most relevant document segments based on mathematical proximity in vector space.
Prompt Design	Strict Grounding Constraint	Directs the model to explicitly refuse answering if the retrieved context is insufficient, preventing hallucinations.

4. Workflow Explanation (LangGraph)

The system utilizes LangGraph to implement a state-driven workflow rather than a linear execution chain. This allows the system to manage complex states and conditional transitions based on the output of specific nodes.

Node Responsibilities

- **Router Node:** The entry node that uses the LLM to categorize the input query into specific categories (Greeting, RAG Query, or Escalation).
- **RAG Node:** The processing node that triggers the search within ChromaDB and passes the results to the LLM for grounded generation.
- **HITL Escalation Node:** A terminal node that stops automation and flags the session for human intervention.

State Transitions

Each node receives the current "AgentState," modifies it (e.g., adding an answer or setting a flag), and passes it back to the graph. The graph then evaluates conditional edges to determine the next destination until the "END" state is reached.

5. Conditional Logic

Intent Detection

Intent is detected via zero-shot classification. The system evaluates the user message for sentiment (frustration), explicit requests (asking for a manager), and semantic relevance (technical question). The LLM returns a structured JSON response identifying the intent.

Routing Decisions

- **Greeting:** Directs to a lightweight greeting node to save computational resources.
- **Technical Query:** Directs to the retrieval module to search the knowledge base.
- **Escalation:** Triggered by keywords like "help," "human," or detected high-frustration scores.

6. Human-in-the-Loop (HITL) Implementation

HITL is the mechanism that allows a transition from automated AI support to a human agent. In this system, it is implemented as a state flag within the graph.

Role of Human Intervention

When the AI encounters a query it cannot resolve with high confidence, or when the user explicitly requests a human, the workflow halts generation. This prevents the bot from providing repetitive or frustrating answers during complex scenarios.

Benefits and Limitations

The primary benefit is increased customer trust and the handling of edge cases that fall outside the provided documentation. The limitation is the requirement for available human staffing to resume the conversation once the AI has been paused.

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed:** Higher Top-K values provide more context but increase LLM processing time. We settled on Top-4 as the optimal balance.
- **Chunk Size vs. Context:** Smaller chunks are more precise but may lose the "big picture." We used a character-based splitter with overlap to mitigate this.
- **Cost vs. Performance:** Groq LPUs were selected to provide enterprise-level speed at a significantly lower cost than traditional GPU cloud hosting.

8. Testing Strategy

Testing followed a rigorous "Query-Response" validation approach using several categories of samples:

- **Direct Knowledge:** "How do I install the software?" - Verified against PDF content.
- **Out-of-Scope:** "What is the weather like?" - Verified that the system refuses to answer.
- **Escalation:** "I am going to close my account, give me a human." - Verified that the HITL flag triggers.

9. Future Enhancements

- **Multi-Document Support:** Expanding metadata filters to allow cross-referencing between multiple unrelated PDFs.
- **Feedback Loop:** Implementing a user-rating system where poor ratings automatically trigger the escalation node for a human review.
- **Memory Integration:** Adding persistent database storage for LangGraph checkpoints to allow for multi-turn conversations across sessions.
- **Deployment:** Moving the current FastAPI architecture into a containerized Docker environment for cloud scaling.